



ALGORITHME ET PROGRAMMATION



Par : *Ing. Ulrich DJOKO*

Ingénieur Informaticien et Expert IT

Email : ulrich_djoko@yahoo.com

Tel : 694 965 770

PARTIE I : INITIATION À L'ALGORITHMIQUE.....	3
CHAPITRE 1 : GENERALITES	3
INTRODUCTION	3
I- NOTION D'ALGORITHME.....	3
1- Définition	3
Exemples	3
2- Structure d'un algorithme	4
3- Propriétés d'un bon algorithme	5
II- Démarche algorithmique	5
Exercices :.....	6
III- Codification	6
1. Processus de programmation.....	7
3. Catégories des langages de programmation	8
IV- III- Eléments de base d'exécution d'un programme	8
1. la mémoire centrale	9
2. le processeur	9
3. les unités d'entrées/sorties.....	9
4. Quelques définitions.....	9
CHAPITRE II : OBJETS ET ACTIONS.....	10
I- LES OBJETS.....	10
1. Les variables	10
a) Caractéristiques d'une variable	10
b) Déclaration des variables	11
c) Les opérateurs de base.....	11
2. Les constantes	12
Exemple d'application	12
II- ACTIONS.....	12
1- L'affectation	12
a) Principe de l'affectation	13
b) Initialisation des variables.....	13
2- Instruction de lecture	14
3- Instruction d'écriture.....	14
Exemple	14
CHAPITRE III : STRUCTURES DE CONTROLE	16
I- Structures conditionnelles.....	16
1. Structure sélective simple (un choix)	16
Finsi	16
Finsi	16
c. Exercice	17

2. Structure alternative (2 choix)	17
Sinon	17
Finsi	17
Sinon	17
Finsi	17
c. Exercice	18
Remarques : Structure alternative imbriquée.....	18
Sinon	18
Sinon	18
3. Structure à choix multiple	19
a. Syntaxe	19
Sinon	19
Fin selon quePrincipe.....	19
Sinon	19
Fin selon que	19
Remarques	20
II- SCHEMAS ITERATIFS	20
1. La structure tant que...faire	20
a. Syntaxe	20
c. Exercices	21
2. La boucle Pour...faire	22
a. Syntaxe	22
3. La boucle répéter...jusqu'à.....	23
a. Syntaxe	23
Conclusion	23

PARTIE I : INITIATION À L'ALGORITHMIQUE

CHAPITRE 1 : GENERALITES

INTRODUCTION

L'une des principales activités de l'informatique est la programmation, dont l'objectif est la production des logiciels.

Programmer consiste à donner un ensemble d'instructions à un automate, qu'il devra exécuter pour atteindre un objectif donné. Dans la pratique, l'écriture d'un programme consiste généralement à implanter une méthode de résolution déjà connue et souvent conçue indépendamment d'une machine. Le terme « algorithme » est employé en informatique pour décrire une telle méthode, et les algorithmes sont l'un des centres d'intérêt de la plupart des domaines de l'informatique.

I- NOTION D'ALGORITHME

1- Définition

Un algorithme est une suite ordonnée d'actions permettant de résoudre un problème précis en un temps fini.

Exemples

1. Une recette de cuisine
2. Une carte
3. Le manuel d'utilisation d'un matériel
4. algorithme de résolution de l'équation

$$Ax^2+bx+c=0, \text{ avec } a, b, c \in \mathbb{R}.$$

Nécessaire

$a, b, c, \text{ delta}, x_1, x_2, x_0$

Actions

- Vérifier que a est non null
- Calculer $\text{delta} = b^2 - 4.a.c$;
- Si $\text{delta} < 0$ alors pas de solution ;
- Si $\text{delta} = 0$ alors $x_0 = -b/2a$;
- Si $\text{delta} > 0$ alors on a deux solutions $x_1 = \frac{-b - \sqrt{\text{delta}}}{2a}$ et $x_2 = \frac{-b + \sqrt{\text{delta}}}{2a}$

2- Structure d'un algorithme

Un langage de définition algorithmique (LDA) utilise un ensemble de mots clés et de structures précises pour décrire les algorithmes. Son avantage est de pouvoir être facilement transcrit dans un langage de programmation, mais aussi de faciliter la communication entre programmeurs.

Les mots clés utilisés dans notre LDA sont :

- ALGORITHME : permet de donner un nom à l'algorithme. Ce nom doit avoir un lien avec le problème à résoudre.
- DEBUT : marque le commencement de l'algorithme.
- FIN : marque la fin de l'algorithme.
- VARIABLE : permet de déclarer les variables.
- CONSTANTE : permet de déclarer les constantes.

La structure générale d'un algorithme est donc la suivante :

Entête	Algorithme	nom de l'algorithme
	Constantes	
Déclaration	Variables	
	Début	
Corps	Action1 ;	
de	Action 2 ;	
l'algorithme	Action n ;	
	Fin	

Remarque

- Le nom de l'algorithme et des variables ne doit pas contenir d'espace.
- Chaque instruction se termine par un point-virgule.
- L'en-tête : cette partie permet de donner un nom à l'algorithme. En général il doit être parlant pour permettre aux lecteurs de comprendre le problème qui est traité.
- Les déclarations : c'est la liste exhaustive des objets manipulés dans le corps de l'algorithme.
- Le corps de l'algorithme : c'est dans cette partie qu'on place toutes les instructions à exécuter pour résoudre le problème posé.

3- Propriétés d'un bon algorithme

- L'algorithme doit pouvoir s'exécuter en un temps fini ;
- L'algorithme ne doit pas dépendre d'un langage de programmation ;
- L'algorithme ne doit pas dépendre d'une machine physique donnée ;
- Il doit se dérouler ou s'exécuter en un temps minimal tout en consommant un minimum de ressource (mémoire processeur) ;
- Il doit faire ressortir clairement les différents objets manipulés.

II- Démarche algorithmique

La démarche à suivre pour obtenir un algorithme de résolution d'un problème donné compte plusieurs étapes :

1. Comprendre clairement le problème posé
 - Qu'est ce qu'il faut faire ? quel est l'objectif à atteindre ?
 - Quelles informations seront fournies à l'algorithme ?
 - Quel sera le résultat de l'algorithme ? Quelles informations seront produites en sortie de l'algorithme ?
2. Essayer de résoudre manuellement le problème sur des exemples concrets
 - ceci permet de bien comprendre ce que devra faire l'algorithme
 - permet de bien comprendre les difficultés du problème traité
 - aide à identifier les étapes de résolution du problème
3. Décrire le principe de résolution du problème.
 - Il s'agit ici de dire comment les données en entrée sont transformées en résultat.
 - on décrit les étapes de résolution à l'aide de courtes phrases en langage naturel ou à l'aide de symboles mathématiques ...
4. Décrire l'environnement algorithmique

C'est la liste des objets nécessaires au processus pour résoudre le problème. Chaque objet de l'environnement est caractérisé par son nom, son type, sa nature, son utilisation, sa valeur initiale, sa valeur finale et son sens. Le tout doit être résumé dans un tableau de la manière suivante.

Nom	Type	Nature	Utilisation	Valeur initiale	Valeur finale	Sens

Nature = variable ou constante

Sens = entrée, sortie, entrée/sortie

5. Ecrire l'algorithme proprement dit.

Remarque

On appelle dossier algorithmique un document qui résume la démarche utilisée pour produire l'algorithme. Il comporte 4 sections:

- ✓ Enoncé du problème: Il s'agit de dire de façon consiste en quoi consiste le problème à résoudre.
- ✓ Principe de résolution: Il s'agit de dire en utilisant le langage naturel ou des symboles mathématiques comment le problème sera résolu.
- ✓ L'environnement
- ✓ L'algorithme proprement dit

NB : La démarche algorithmique est souvent appelée phase d'analyse du problème

Exercices :

Proposer un dossier algorithmique pour chacun des problèmes suivants :

1. calcul de la surface d'un cercle
2. recherche d'un voleur dans une liste de prospects.
3. résolution d'une équation du 1^{er} degré.
4. On considère une pile de dossiers classés par ordre alphabétique sur le nom. Donner un algorithme qui détermine si une personne de nom X a bien un dossier dans la pile.
5. Reconnaître si un mot est palindrome ou pas.

III- Codification

Une fois qu'on trouve le moyen de passer des données au résultat, il faut être capable de rendre cette solution compréhensible par l'ordinateur.

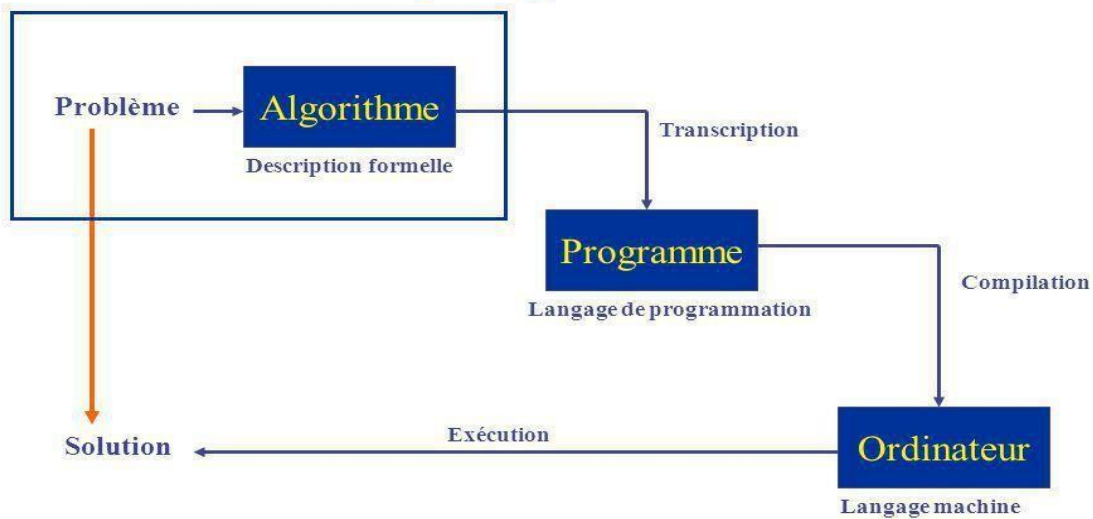
Le langage utilisé par l'ordinateur (langage machine) est très difficilement compréhensible par l'homme. C'est pourquoi en pratique on utilise des langages intermédiaires qu'on appelle langages de programmation. Ces langages nécessitent une transformation avant d'être exploitable par la machine.

La codification est l'activité qui consiste à traduire un algorithme dans un langage de programmation dans le but d'obtenir un programme.

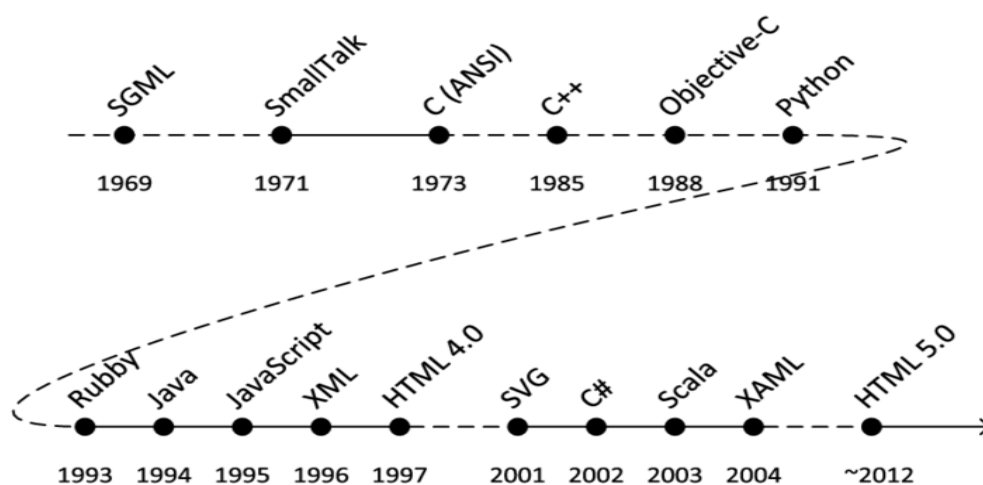
Un programme est une suite d'instructions que la machine va lire et exécuter au fur et à mesure jusqu'à ce qu'elle rencontre l'instruction de fin de programme.

1. Processus de programmation

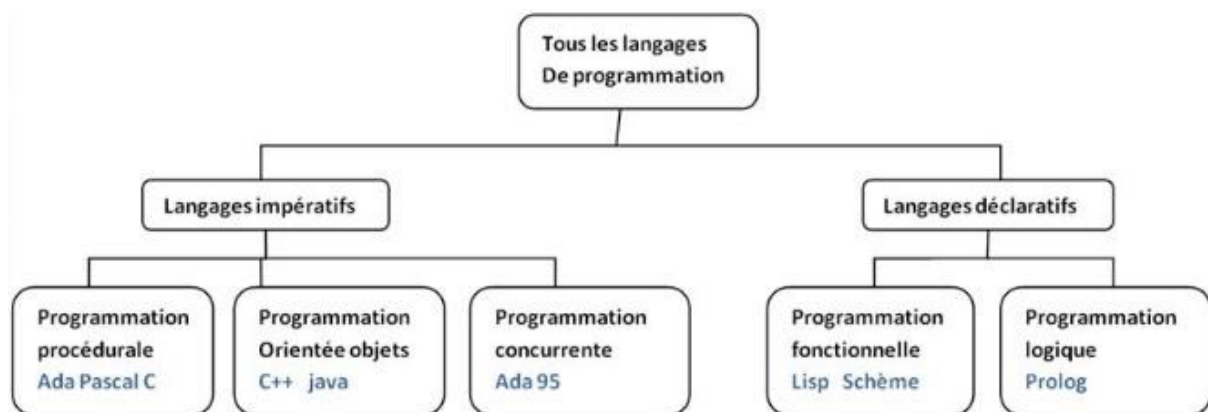
Étapes de la construction d'un programme



2. Historique des langages



3. Catégories des langages de programmation

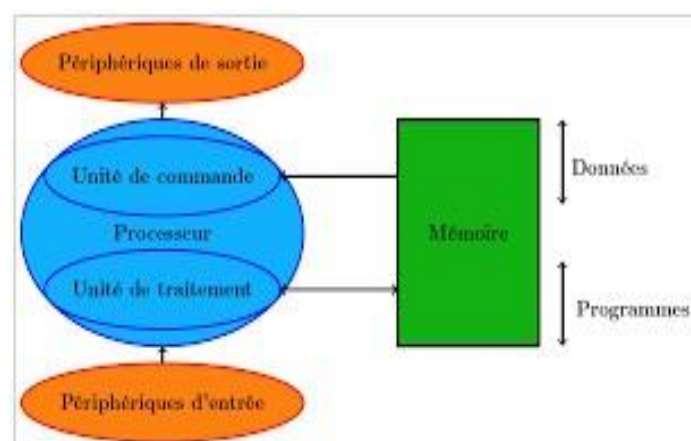


4. Types d'exécution des langages de programmation

En fonction de leur mode d'exécution par la machine, on distingue plusieurs catégories de langages de programmation : *les langages compilés*, *les langages interprétés*, les langages intermédiaires, les langages transcompilés.

IV- III- Eléments de base d'exécution d'un programme

Trois composants de l'ordinateur entrent en jeu dans l'exécution d'un programme informatique : le processeur, la mémoire centrale et les unités d'entrées/sorties.



1. la mémoire centrale

C'est le composant de l'ordinateur qui contient les données et les instructions des programmes en cours d'exécution. A l'exécution d'un programme, ses données et ses instructions sont stockées en mémoire centrale. Les données stockées en mémoire centrale sont appelées variables.

2. le processeur

C'est le composant responsable de l'exécution des programmes. Il lit les différentes instructions du programme en mémoire et les exécute au fur et à mesure, en utilisant les données stockées en mémoire centrale.

3. les unités d'entrées/sorties

Ce sont des éléments qui assurent la communication entre l'utilisateur et la machine. Elles permettent aux usagers de fournir des données au programme et inversement de recevoir les résultats des traitements par le programme.

Exemples : clavier, souris, imprimante

4. Quelques définitions

Algorithmique : discipline qui étudie les algorithmes : leurs conception, leurs performances (complexité), leurs validité.

Programme: ensemble d'instruction compréhensible et exécutable par un processeur.

Logiciel : c'est ensemble de programmes permettant à l'ordinateur de résoudre un problème précis.

Langage de programmation : ensemble de règles et de syntaxes permettant de traduire un algorithme sous une forme compréhensible par un ordinateur : JAVA, C++, Visual Basic, python...

Géni-logiciel : discipline de l'informatique qui étudie les méthodes, les outils et les techniques permettant de produire des logiciels de qualité.

Compilation : c'est la traduction d'un programme source en programme exécutable.

Interpréteur : c'est un logiciel spécifique qui traduit ou exécute les instructions d'un programme source.

CHAPITRE II : OBJETS ET ACTIONS

Dans un programme informatique, on rencontre deux principaux concepts : les objets (variables) et les actions (instructions).

Les objets représentent l'information traitée par le programme alors que les actions correspondent aux opérations que la machine doit effectuer pour transformer les données entrées en résultats.

I- LES OBJETS

Un objet désigne toute partie identifiable d'une information en cours de traitement. Il est caractérisé par un nom, un type et une valeur.

Les principaux objets manipulés en programmation sont les variables et les constantes.

1. Les variables

Les programmes ont pour but de traiter les données à fin de produire les résultats. Pour cela les données d'un programme doivent être stockées en mémoire centrale pour un traitement par le microprocesseur.

Une variable est espace mémoire réservée au stockage d'une donnée au cours du traitement par le microprocesseur. Chaque variable possède un nom unique appelé **identificateur**, par lequel on peut accéder et manipuler son contenu. La valeur d'une variable peut varier au cours de l'exécution d'un programme ; les anciennes étant remplacées par les nouvelles. Une variable ne peut donc pas avoir plusieurs valeurs au même moment.

a) Caractéristiques d'une variable

Les trois principales caractéristiques d'une variable sont :

- L'identificateur (le nom de la variable)

Un identificateur peut être composé de lettre et de chiffre. Mais il ne peut pas commencer par un chiffre et ne peut contenir d'espace.

L'identificateur d'une variable doit être suffisamment significatif (parlant) pour qu'on reconnaisse sa fonction aisément. Par exemple pour une variable représentant le prix d'un produit, on évitera d'utiliser le nom **a** mais plutôt **prix**.

- Le type : les opérations sur les variables dépendent de leur type. Le type d'une variable définit donc les opérations possibles sur cette dernière.
- La valeur : la valeur est incluse dans le domaine de définition du type de la variable. Elle représente le contenu de la variable à un moment donné.

b) Déclaration des variables

Pour qu'un programme puisse utiliser une variable, il faut au préalable la déclarer. La déclaration d'une variable permet au système de réserver de l'espace mémoire pour cette variable.

La déclaration d'une variable doit indiquer son identifiant et son type.

Exemple : variable prix : réel ;
 age : entier ;
 nom : chaîne ;

c) Les opérateurs de base

Types	Exemples	Opérations	Symboles
Réel	-3.6 ; 5.3 ; 2.5 ; 0.12 ; 32.7 ; 36.7 ;	Addition ; Soustraction ; multiplication ; division ; comparaisons.	+ ; - ; * ; / ; < ; > ; <= ; >= ; = ; ≠
Entier	-3 ; 4 ; 10	Addition ; soustraction ; division ; comparaison ; modulo ; multiplication ; division entière	+ ; - ; * ; / ; < ; > ; <= ; >= ; = ; <> , %, div
Booléen	Vrai ; Faux	Comparaison ; Conjonction ; disjonction ; négation	= ; ≠ ; non ; ET ; OU
Chaîne de caractères	"bonjour"	Comparaison ; Concaténation	< ; > ; <= ; >= ; ≠ ; = ; + ; · ;
caractères	"a" ; "b" ; "c"	comparaison	< ; <= ; > ; >= ; = ; ≠

Tableau de vérité

A	B	Non (A)	A et B	A ou B
Vrai	Vrai	faux	Vrai	Vrai
Vrai	Faux	Faux	Faux	Vrai
faux	Vrai	Vrai	Faux	Vrai
faux	Faux	Vrai	Faux	Faux

2. Les constantes

Contrairement aux variables, les constantes désignent des références à des contenus qui ne changent pas pendant l'exécution du programme.

La syntaxe de déclaration d'une constante est la suivante :

Constante non_constant = valeur ;

Exemple PI = 3.14 ;

Exemple d'application

Evaluez et donnez le type de chacune des expressions suivantes :

- a- $(2*3-5/2)*0.8$;
- b- Vrai ET $(6<14)$;
- c- $((2*5-9) < 11)$ OU $(3<4)$.

II- ACTIONS

Les actions d'un programme portent sur les objets et ont pour but de transformer cette dernière. En algorithmique il existe trois actions de base : l'affectation, la lecture et l'écriture.

1- L'affectation

Elle consiste simplement à placer une valeur dans une variable (ce qui revient à changer le contenu de la variable). La nouvelle valeur est évaluée à partir d'une nouvelle expression qui peut être :

- Soit une autre variable ou une constante ;
- Soit une valeur littérale ;
- Soit une expression qui doit être évaluée.

En algorithmie, pour représenter l'affectation on utilise le symbole 

Exemple variable x, y : entier ;

x ← 2 ; // la valeur de x devient 2

y ← x+3 ; // la valeur de y devient 5

x ← y ; // la valeur de x devient 5

a) Principe de l'affectation

Une instruction d'affectation est formée de deux parties :

- A gauche du symbole d'affectation on trouve toujours des variables destinées à recevoir une valeur ;
- A droite, on trouve une expression qui doit être évaluée et dont le résultat doit être affecté à la variable de gauche.

Remarque : Les parties de gauche et de droite doivent être de même type.

b) Initialisation des variables

Au début d'un programme, les variables n'ont pas encore reçu des valeurs : on dit qu'elles sont indéfinies. La plus part du temps, une variable doit être initialisée c'est-à-dire recevoir une valeur initiale avant son utilisation ; sinon le contenu de la variable sera indéfini et le comportement du programme sera aléatoire.

Une bonne habitude consiste à initialiser les variables au début du programme.

Exemple d'application

Donner la valeur de A et B à la fin de l'exécution de l'algorithme suivant :

Algorithme affecté ;

Variable a, b : entier ;

Début

a ← 3 ;

b ← a+2 ;

a ← b*3-7 ;

b ← b+1-a ;

a ← a ;

Fin

2- Instruction de lecture

Un programme qui ne communique pas avec son environnement externe est complètement inutile. Deux principales instructions permettent à un programme de communiquer avec son environnement : **la lecture et l'écriture**.

L'instruction de **lecture** permet de récupérer une valeur se trouvant sur un périphérique(le clavier) pour l'envoyer en mémoire centrale dans une variable. Tant que l'utilisateur n'entre rien au clavier, le déroulement du programme est bloqué.

Remarque : l'entrée d'une valeur au clavier consiste à saisir cette valeur puis à taper sur la touche Enter.

Syntaxe : saisir/lire (variable1, [variable2,...variable n]) ;

Exemple :

- lire (x) ; cette instruction va lire la valeur saisie au clavier puis afficher la valeur à x.
- Lire(x, y) ; elle va lire deux valeurs saisies au clavier puis afficher la première valeur à x et la seconde à y.

3- Instruction d'écriture

L'instruction d'écriture ou d'affichage permet au programme d'afficher des messages ou des résultats à l'utilisateur au travers d'un périphérique de sortie (l'écran).

Syntaxe : Ecrire (expression1, [expression 2 ... expression N]) ;

Exemple

- Ecrire ("bonjour") ; cette instruction affiche le texte **bonjour** à l'écran.
- Ecrire (nom) ; elle va afficher la valeur de la variable nom. Si nom vaut papa : on affichera **papa** à l'écran.
- Ecrire (nom, prénom) ; elle permet d'afficher les valeurs des variables nom et prénom. Si nom= bonjour et prénom= bonsoir : on aura à l'écran **bonjourbonsoir**.

Remarque : on peut mélanger l'affichage des valeurs littérales et des variables. Cela est particulièrement utile si on veut voir apparaître un libellé (texte accompagnant la saisie des données ou bien l'édition des résultats permettant de guider l'utilisateur).

Exemple : Ecrire ("voici les résultats : x=", x, "et y=", y) ;

Si x= 10 et y= 5 alors on aura à l'écran : voici les résultats : x=10 et y= 5.

Exercice d'application :

1. Ecrire un algorithme qui demande à l'utilisateur de saisir deux valeurs entières puis calcul et affiche la somme de ces deux variables.
2. Ecrire un programme qui calcul et affiche la surface d'un carré.
3. Ecrire un programme qui demande l'utilisateur d'entrer son nom, puis affiche bonjour suivie de son nom.
4. Ecrire un programme qui calcul et affiche le périmètre et la surface d'un rectangle.
5. Ecrire un programme qui calcul et affiche le salaire d'un employé sachant que $\text{salaire} = 500\,000 + \text{nombre d'enfants} * 10\,000 + \text{nombre d'heures de travail} * 500$.

CHAPITRE III : STRUCTURES DE CONTROLE

Dans un programme informatique, l'exécution des instructions se fait rarement de façon séquentielle. Dans certaines situations, le programme doit prendre un chemin d'exécution plutôt qu'un autre en fonction du contexte. Les structures de contrôle permettent au programmeur de définir les différents chemins d'exécution d'un programme. On classe les structures en deux catégories : les structures conditionnelles et les structures itératives (ou boucles).

I- Structures conditionnelles

1. Structure sélective simple (un choix)

a. Syntaxe

Si (condition) **Alors**

Instruction ;

Finsi

Si l'expression « condition » est évaluée à **vrai** alors le bloc d'instruction est exécuté, sinon il est ignoré.

b. Exemple

Algorithme maximum

Variable a, b, max : réel ;

Début

Ecrire ("entrer les valeurs de a et b") ;

Lire (a, b) ;

max ← a ;

Si (a<b) **Alors**

Max ← b ;

Finsi

↓ Ecrire (" maximum =", max) ;

Fin

c. Exercice

Ecrire un programme qui prend en entrée un entier et affiche le message « erreur de saisie » si ce nombre est plus petit que 0.

2. Structure alternative (2 choix)

a. Syntaxe

Si (condition) **Alors**

Bloc Instruction 1 ;

Sinon

Bloc Instruction 2 ;

Finsi

Si l'expression « condition » est évaluée à vrai alors le bloc d'instruction 1 est exécuté et le bloc d'instruction 2 est ignoré ; sinon le bloc d'instruction 1 est ignoré et le bloc d'instruction 2 est exécuté.

b. Exemple

Algorithme maximum

Variable a, b, max : réel ;

Début

Ecrire ("entrer deux réels") ;

Lire (a, b) ;

Si (a<b) **Alors**

max ← b ;

Sinon

max ← a ;

Finsi

Ecrire (" maximum =", max) ;

Fin

c. Exercice

Ecrire un programme qui prend en entrée un entier et affiche en sortie « nombre négatif » s'il est inférieur à zéro et « nombre positif » s'il est supérieur à zéro.

Remarques : Structure alternative imbriquée

Il est possible d'introduire une structure alternative à l'intérieur d'une autre : dans ce cas, on parle de **structure alternative imbriquée**.

Exemple : Ecrivons un programme qui prend en entrée un entier puis affiche « nombre positif » ou « nombre négatif » ou « nombre nul » en fonction du signe de N.

Algorithme signe_nombre

Variable N : entier ;

Début

Ecrire ("entrer un nombre entier") ;

Lire (N) ;

Si (N<0) **alors**

Ecrire ("nombre négatif") ;

Sinon

Si (N>0) **alors**

Ecrire ("nombre positif") ;

Sinon

Ecrire ("nombre nul") ;

Finsi

Finsi

Fin

Remarque :

- Il n'y a pas de point-virgule à la fin de l'instruction qui précède le **Else**.
- Le **sinon** se rapporte toujours au dernier **Si** encore ouvert.

3. Structure à choix multiple

Lorsque l'imbrication des alternatives devient importante, l'utilisation de la structure à choix multiple peut être utile pour écrire un code plus propre. Cette structure est très utilisée pour réaliser les menus d'une application.

a. Syntaxe

Selon que (expression) **vaut**

Valeur 1 : instruction 1 ;

Valeur 2 : instruction 2 ;

:

Valeur n : instruction n ;

Sinon

Autre instruction ;

Fin selon que

Principe

Si l'expression est validée à l'une des valeurs (valeur 1, valeur 2, ..., valeur n) alors c'est le bloc d'instruction correspondant à cette valeur qui sera exécuté. Si l'évaluation de l'instruction ne prend aucune instruction parmi ces valeurs alors c'est le bloc autre instruction qui sera évalué.

b. Exemple

Algorithme nom_chiffre

Variable n : entier ;

Début

Ecrire ("entrer un nombre entier entre 0 et 2") ;

Lire (N) ;

Selon que (N) **Vaut**

0 : écrire ("zéro") ;

1 : écrire ("un") ;

2 : écrire ("deux") ;

Sinon

Ecrire ("erreur de saisie") ;

Fin selon que

Fin

Remarques

- Expression peut être évaluée à une valeur de type entier, caractère, booléen ou énuméré. Mais pas un réel ou une chaîne de caractère.
- Les val 1, val 2, ..., val n sont des constantes du même type qu'une expression.

II- SCHEMAS ITERATIFS

Les schémas itératifs ou structures répétitives encore appelés boucles permettent de répéter automatiquement un traitement (c'est-à-dire une instruction simple ou composée) autant de fois qu'il est nécessaire. Il existe trois grands types de structures répétitives : la structure **POUR**, la structure **Tant que...faire** et la structure **Répéter... jusqu'à**.

Seule la boucle Tant que est fondamentale car elle permet de réaliser toutes les autres boucles.

1. La structure tant que...faire

Elle permet de répéter un traitement tant que la condition est vraie.

a. Syntaxe

Tant que (condition) Faire

Instruction(s) ;

Fin tant que

Lorsque dans le programme on rencontre une boucle tantque :

- alors on évalue condition ; si la valeur de condition est vraie, on exécute le bloc d'instruction jusqu'à rencontrer le fin tantque.
- On retourne ensuite sur la ligne du tantque et on procède au même examen et ainsi de suite.
- La boucle ne s'arrête que lorsque la condition prend la valeur fausse ; dans ce cas le programme poursuit son exécution après fin tantque.

b. Exemple : calculer la somme des N premiers nombres naturels.

Algorithme somme

Variables s, i, n : entier ;

Début

 Ecrire ("entrer la valeur de n") ;

 Lire (n) ;

 s 0 ;

 i 1 ;

 Tantque (i<n) faire

 S s + i ;

 I i + 1;

 FinTantque

 Ecrire ("somme=",s) ;

Fin

Remarque :

- Dans la boucle tant que, la condition est vérifiée avant l'exécution du corps de la boucle, ce qui est un gage de sécurité.
- Si on a une structure tant que dans laquelle la condition ne prend jamais la valeur « Faux » alors on dit qu'on a une boucle infinie.

c. Exercices

- a.)Ecrire un programme qui demande à un utilisateur d'entrer un nombre entier puis calcul et affiche le produit des n premiers nombres entier naturel non nul.
- b.)Ecrire un programme qui affiche la table de multiplication d'un nombre entré au clavier par l'utilisateur.
- c.) Ecrire un programme qui demande à l'utilisateur de saisir un entier strictement supérieur à 100 jusqu'à ce que la valeur convienne. En cas d'erreur le programme affiche le message « erreur de saisie veuillez recommencer SVP ! ». Lorsque l'utilisateur entre la bonne valeur, on affiche le message « valeur correcte ».

2. La boucle Pour...faire

Cette boucle permet de répéter une liste d'instructions, un nombre de fois connu.

a. Syntaxe

```
Pour compteur de début à fin faire  
    Instruction ;  
Finpour
```

```
Ou  
Pour compteur de début à fin pas n faire  
    Instruction ;  
Finpour
```

- La variable compteur est de type entier.
- Elle est initialisée à la valeur « début » et augmente automatiquement à chaque tour de boucle jusqu'à la valeur « fin ».
- Pour chaque valeur prise par la variable « compteur », le corps de la boucle est exécuté.
- Lorsque la boucle prend la valeur « fin' », le corps de la boucle est exécuté une dernière fois puis le programme sort de la boucle.

Remarque

- Par défaut la variable est incrémentée de 1 à chaque tour de boucle.
- Pour modifier la valeur d'incrémentation, il suffit de rajouter le mot « pas » (step) suivie de la valeur correspondante.

b. Exemple : écrire un programme qui affiche la table de multiplication d'un entier entré au clavier.

Algorithme table multiplication

Variable i, n : entier ;

Début

Ecrire ("entrer un nombre entier") ;

Lire (n) ;

Pour i de 1 à 20 faire

Ecrire (n, "*", i, "=", n*i);

Finpour

Fin

3. La boucle répéter...jusqu'à

Cette boucle sert à répéter un traitement jusqu'à ce qu'une certaine condition soit remplie.

a. Syntaxe

Répéter

Instruction (s) ;

Jusqu'à (condition) ;

b. Exemple : Ecrire un programme qui permet de saisir un entier positif jusqu'à ce que la valeur convienne.

Conclusion

- Si le nombre d'itération est connu a priori on utilise la boucle Pour.
- Sinon on utilise la boucle tantque (quand le nombre d'itération peut être nul) ou bien la boucle Répéter (quand il y a toujours au moins une itération).

Exercice : écrire un programme permettant d'élever un nombre à la puissance n.